

## **CSC420 Assignment 3**

Xintong Zhang 1006668660

### **Part I**

#### **Q1 reparameterization trick**

The reparameterization trick in a variational autoencoder (VAE) helps us "detach" randomness from the network's learning process. By rewriting the random sampling step as a fixed noise input adjusted by learned parameters, the model can backpropagate through this step, allowing it to learn effectively while still generating varied samples.

#### **Q2 GAN**

Calculating the loss function requires a `detach()` on the generator's output. This is because we don't want gradients to flow back into the generator when updating the discriminator, ensuring that each model's parameters are updated independently.

#### **Q3 VQ-VAE**

$\text{line quantized} = \text{inputs} + (\text{quantized} - \text{inputs}).$

`detach()` replaces inputs with the nearest quantized vectors while preserving gradients. By detaching  $(\text{quantized} - \text{inputs})$ , this operation allows gradients to flow through inputs during backpropagation but not through quantized, ensuring that only the encoder is updated, while quantized remains fixed.

#### **Q4 FID**

FID score measures the quality and diversity of images generated by GANs by comparing them to real images. It calculates the distance between the feature distributions of generated and real images, as extracted by a pretrained Inception network. FID treats these features as multivariate Gaussians and computes the Fréchet distance between the two distributions, considering both the mean and covariance. Lower FID scores indicate that the generated images are closer in quality to real images, with high similarity and diversity. This metric captures perceptual quality more accurately than simpler metrics like mean squared error.

### Q5 Corner Detection

$$1. \quad N - \lambda I = \begin{pmatrix} I_x^2 - \lambda I & I_x I_y \\ I_x I_y & I_y^2 - \lambda I \end{pmatrix}$$

$$\begin{aligned} \det(N - \lambda I) &= (I_x^2 - \lambda I)(I_y^2 - \lambda I) - I_x I_y * I_x I_y \\ &= I_x^2 I_y^2 - \lambda I_x^2 - \lambda I_y^2 - I_x^2 I_y^2 \\ &= \lambda^2 - \lambda(I_x^2 + I_y^2) \\ &= 0 \end{aligned}$$

$$\lambda_1, \lambda_2 = 0, I_x^2 + I_y^2$$

eigenvalues are  $\lambda_1 = 0$   $\lambda_2 = I_x^2 + I_y^2$

2. Need to show  $uMu^T \geq 0$  for any vector

From part 1, both eigenvalues of  $N$  is non-negative

So  $N$  is positive semi-definite

$uNu^T \geq 0$ , for any vector  $u$

Therefore,

$$\begin{aligned} uMu^T &= u \sum_x \sum_y w(x,y) N(x,y) u^T \\ &= \sum_x \sum_y w(x,y) uN(x,y) u^T \end{aligned}$$

Since  $w \geq 0$ ,  $uN(x,y)u^T \geq 0$ , then each term is non-negative

So  $uMu^T \geq 0$  for any  $u$

Therefore,  $M$  is positive semi-definite.

## Q6 Optical Flow

### 1. In homogeneous image areas

Yes

Optical flow relies on detecting changes in pixel intensity across frames. In homogeneous areas, there is little to no variation, so it is difficult to determine motion or direction accurately.

### 2. In textured image areas

No

Textured areas provide sufficient detail for the algorithm to track changes in intensity patterns between frames. Thus, this enables accurate optical flow estimation.

### 3. At image edges

Yes

At the image edge, like the side of a building, it often only shows movement in one direction. Optical flow can track motion perpendicular to the edge but struggles to detect any movement along the edge itself.

### 4. At the boundaries of a moving object

Yes

At the boundaries of a moving object, there are sudden changes in motion. The occlusion and appearance changes lead to ambiguity and difficulty in tracking flow.

### 5. Corner of a non-moving object

No

Corners are usually distinctive features with unique intensity gradients in multiple directions, allowing optical flow algorithms to detect small changes effectively.

## Q7 LSTM

**Input Gate:** The input gate should be set to 1. This ensures that each input at every time step is added to the cell state without any reduction or scaling down, which allows an accumulation of inputs over time.

**Forget Gate:** The forget gate should be set to 1 as well. This keeps the previous cell state value intact, meaning no past information is "forgotten" at each time step. The cell state continues accumulating the inputs without losing any prior information.

## Q8 GAN training (non-saturating generator cost)

1. Early in the training, value of  $D(G(z))$  is closer to 0

Early in the training, generator  $G$  produces samples that are not realistic, and discriminator  $D$  can easily distinguish these fake samples from real ones since the generator has not yet learned to generate realistic data. Thus, value of  $D(G(z))$  which represents the probability of the discriminator classifying generated samples as real, is low (closer to 0). This low  $D(G(z))$  value reflects the discriminator's confidence that the generated samples are indeed fake.

2.  $J_1(G) = -\frac{1}{m} \sum_{i=1}^m [\log(D(G(z_i)))]$  is typically preferred.

$J_1(G)$ , the non-saturating generator loss can help address the vanishing gradient problem that may occur with the standard GAN loss ( $J_2(G)$ ). When using  $J_2(G)$ , if the discriminator is too good,  $1-D(G(z))$  is close to 0, and this causes gradients to be small and slows down generator learning.  $J_1(G)$  provides strong gradients even when the discriminator confidently identifies fake samples, as the log function in  $J_1(G)$  grows more slowly, keeping the gradient active and preventing it from vanishing. As a result, the generator can continue improving, even when the discriminator is effective at distinguishing real from fake samples, which is especially beneficial in early training stages.

3. "A GAN is successfully trained when  $D(G(z))$  is close to 1"

False

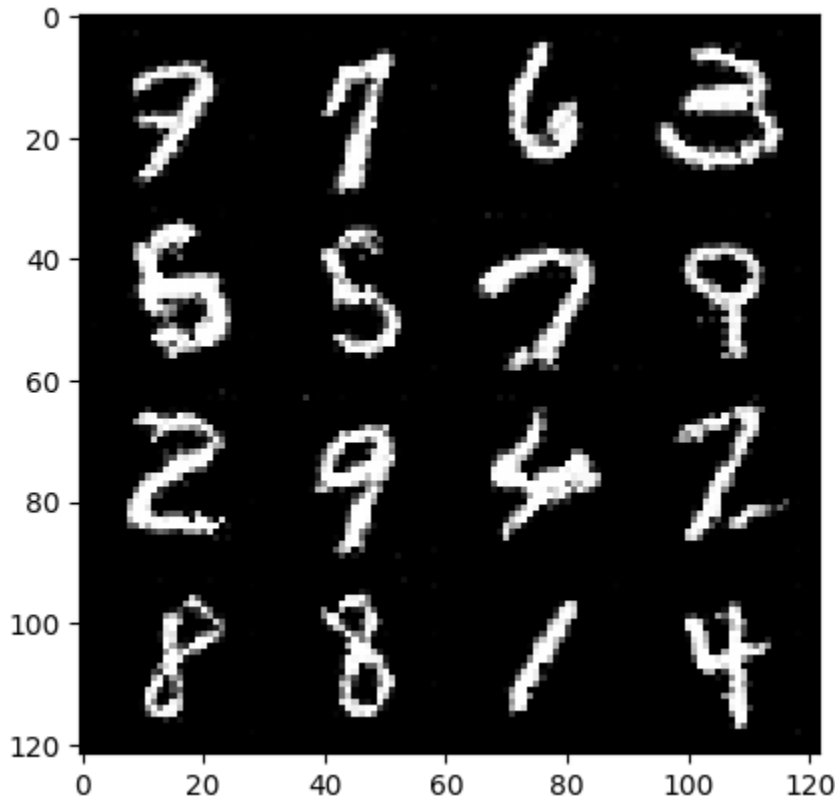
A successfully trained GAN means that the generator has learned to produce samples that are indistinguishable from real data, so  $D(G(z))$  should be around 0.5 instead of close to 1. 0.5 indicates that the discriminator is unable to confidently distinguish between real and fake samples, which means the generator has successfully learned to produce realistic outputs. If  $D(G(z))$  were close to 1, this means that the discriminator is very confident that the generated samples are real. However, in an ideal GAN scenario, the discriminator should not be confident one way or the other.

## Part 2

### Task 1

Result after epoch 20

Epoch: 20 | D\_loss: 0.6916 | G\_loss: 0.7043



MLP GAN (from Tutorial I): requires more epochs to start generating recognizable digits. Based on my observation, around 1000 epochs have general view of digits. Early epochs produce noisy images that are not resemble digits and improvement is gradual. The lack of convolutional layers means it doesn't capture spatial hierarchies effectively.

Convolutional GAN: Begins to produce realistic MNIST digits within just a few epochs. The convolutional layers efficiently learn spatial features, leading to faster convergence. It generates more detailed images early in training. Based on my training, we can see general view of digits within 10 epochs.

### Task 2

1. Similar with Convolutional GAN we perform in Task1, we can see general view of digits within few epochs which is much faster than the MLP GAN.
2. The WGAN's use of the Wasserstein loss improved the gradient flow during training, addressing the instability and vanishing gradient problems often encountered in

traditional GANs. Unlike the binary cross-entropy loss used in the simple and convolutional GANs, the Wasserstein loss measures the Earth-Mover (EM) distance, which provides more meaningful gradients and stabilizes training.

Another difference is the enforcement of the Lipschitz constraint in WGANs through weight clipping. This step ensures the critic function remains within the 1-Lipschitz condition, which is essential for the proper functioning of the Wasserstein distance.

The WGAN also replaces the discriminator with a critic, which evaluates the Wasserstein distance instead of classifying data as real or fake. This fundamental shift results in more stable training and reduces issues like mode collapse.

In general, WGAN outperformed both the simple GAN and the convolutional GAN. It generated higher-quality images with greater diversity and fewer artifacts. It is also stable with fewer required tweaks and adjustments.

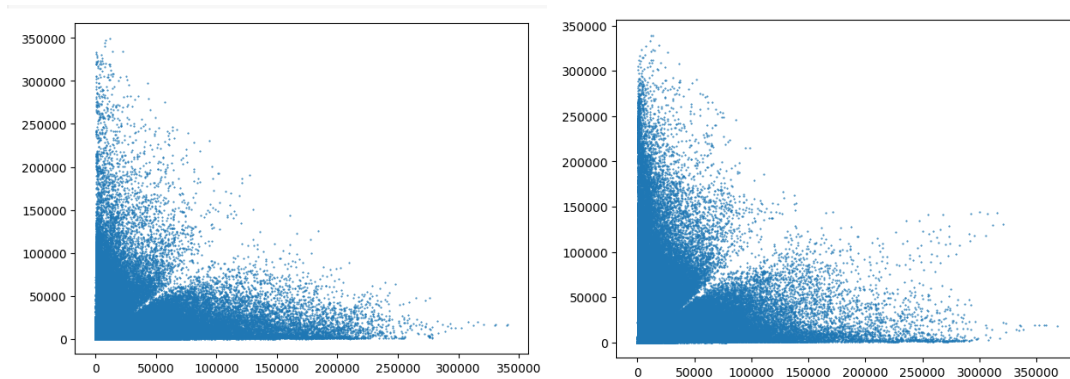
### Task 3

1. eigenvalues are calculated in code with shape

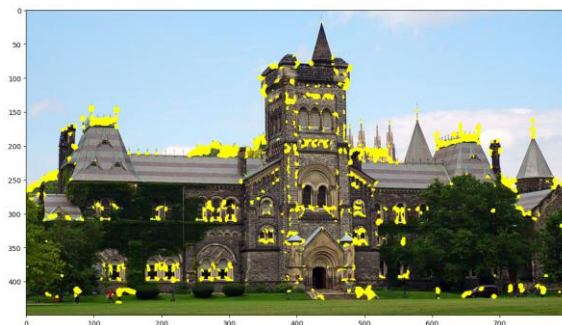
e1 (450, 800, 2)

e2 (533, 800, 2)

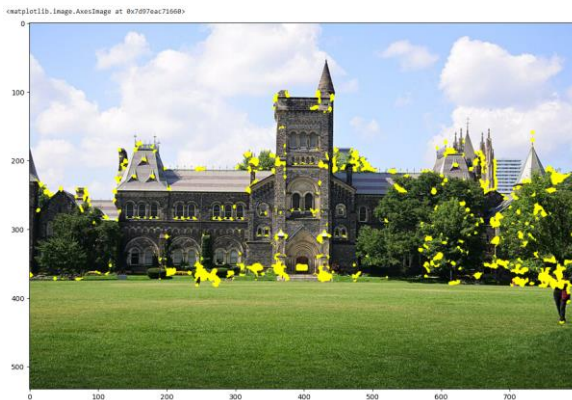
2. scatter plot



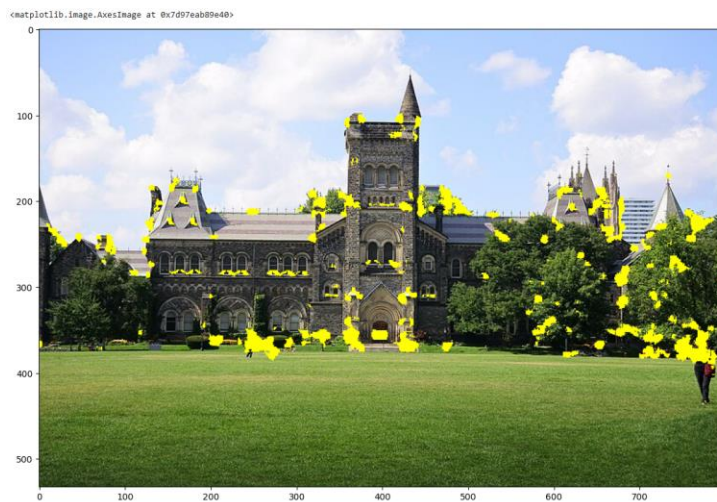
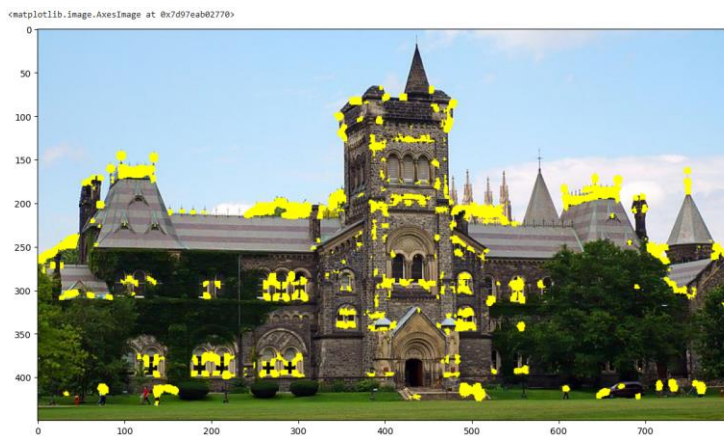
3. threshold =  $2e4$



Threshold =  $3e4$



4. (same threshold as 3)



Increasing the sigma value in the Gaussian kernel for corner detection influenced each of the images by reducing the number of detected corners, especially in areas with finer details.

Image 1: With the higher sigma, the corner detection focused more on larger architectural features, like the main corners and edges of the building, rather than detecting every small texture or detail.

Image 2: Similarly, in Image 2, the increased sigma reduced the number of detected corners in the lawn and trees, where there might have been more texture-based detections with a smaller sigma. The detection primarily highlighted larger structures, such as the main edges of the towers and building outline, while filtering out less significant features in the scene.

Overall, this choice of a larger sigma reduced the sensitivity to small, local variations, focusing the corner detection on more prominent, structural elements in both images and filtering out finer details that might contribute to noise in corner detection.